FleetTrack — Informe final de calidad y decisión GO / NO-GO

Curso 048 Aseguramiento de la Calidad del Software, Universidad Mariano Gálvez. Consolidado por Alejandro con los aportes de Manuel y Andrea. Fecha de las mediciones: 2026-11-05. Ambiente: QA local con Docker Compose y emulador Android 15 (API 35). Commit de referencia: rama `alejandro` sobre `main` (PR #16).

1. Resumen ejecutivo

FleetTrack funciona de punta a punta: dashboard web, API, mensajería con outbox, consumidor idempotente y app Android con modo sin conexión. Las **77 verificaciones automatizadas pasan** y la app Android se validó en un emulador real. El Quality Gate de SonarQube está en **OK** (0 bugs, 0 vulnerabilidades) con cobertura medida del backend de **82,1 %**.

Recomendación: NO-GO para el despliegue final por ahora; el candidato `v0.9.0-rc1` es apto en funcionalidad. El plan establece que un reporte obligatorio ausente deja la ejecución incompleta y sin aprobación final. Faltan cinco evidencias que no dependen de código (sección 8). Con ellas cerradas la decisión pasa a GO sin cambios en el producto.

2. Resultados de pruebas

Nivel	Herramienta	Resultado
Unitarias	xUnit	6 / 6
Integración y API	xUnit contra el stack real	22 / 22
Recuperación ante fallos	`tests/recovery/run-all.sh`	26 / 26
Funcionales, regresión y accesibilidad web	Cypress (+ axe-core)	15 / 15
Móvil	Maestro sobre emulador	8 / 8
Manual Android	Entrega fallida, mensaje sin conexión, evidencia	Aprobado
Total automatizado		**77 / 77**

Las suites de integración y recuperación también se ejecutaron con la API instrumentada para medir cobertura, con el mismo resultado (22/22 y 26/26).

3. Seguridad

Medida	Resultado
Semgrep (sustituto de Fortify): backend, frontend, mobile	0 hallazgos en los tres
SonarQube: bugs / vulnerabilidades / hotspots	0 / 0 / 0
OWASP ZAP web (baseline)	0 FAIL, 3 avisos informativos aceptados
OWASP ZAP API	0 FAIL, 0 avisos
npm audit y NuGet (`--vulnerable`)	0 vulnerabilidades
OWASP Dependency-Check	**Bloqueado**: exige clave de la API de NVD
Contenedores	Todos sin privilegios

Autorización: el conductor no puede operar rutas ni entregas ajenas (pruebas de integración) y los listados de administración devuelven 401 sin token y 403 para el rol conductor.

4. Calidad de código

Medida	Valor
Quality Gate	**OK**
Cobertura del backend (unitarias + integración + recuperación)	82,1 %
Bugs / vulnerabilidades / code smells	0 / 0 / 4 (aviso informativo de logging CA1873)
Duplicación	0 %

La cobertura de código se mide sobre el backend (C#, xUnit). Quedan fuera del cálculo, y la decisión consta en `tests/coverage/sonar-scan.sh`: las migraciones de EF (código generado), el frontend y la app móvil (su interfaz se valida con Cypress, axe y Maestro, no con cobertura de líneas) y los scripts, documentos e infraestructura. El umbral del Quality Gate no se modificó. La condición de cobertura de código nuevo no tiene líneas que medir en este análisis (el código nuevo es de pruebas y herramientas), por lo que no bloquea; la cobertura global del backend (82,1 %) supera el 80 % del umbral aunque esa condición no se evalúe sobre código nuevo.

5. Rendimiento

Corridas cortas con k6 sobre `POST /api/tracking` (todas con 0 % de errores):

Escenario	Usuarios virtuales	p95	Errores
Load	20 (30 s)	67,5 ms	0 %
Stress	20 → 60	81,2 ms	0 %
Spike	20 → 80 → 20	20 ms	0 %

Pendiente: la escala del plan (100 usuarios durante 10 min, stress hasta 300 y spike de 100 a 300) no se ejecutó; el Jenkinsfile la dispara con el parámetro `FULL_PERFORMANCE`. Mientras tanto los resultados cortos no demuestran el criterio de salida.

6. Defectos

ID	Descripción	Severidad	Estado
D-01	Android 9+ bloqueaba HTTP: no había login móvil	Alta	Corregido (`network_security_config`)
D-02	La cámara de evidencia no abría (faltaba `<queries>`)	Alta	Corregido
D-03	Mensaje engañoso al finalizar sin red	Media	Corregido
D-04	Experimento E2 inestable tras otras suites	Media	Corregido
D-05	Sonar: 2 bugs, 5 vulnerabilidades, 14 smells	Media	Corregido
D-06	ZAP: 10 avisos de cabeceras	Media	Corregido (3 informativos aceptados)
D-07	Flujos Maestro con teclado, cámara y patrones incorrectos	Baja	Corregido
D-08	El mensaje "guardada" no se limpia tras sincronizar	Baja	**Abierto** (cosmético)
D-09	Quality Gate en ERROR por cobertura no medida	Alta	Corregido: cobertura medida con `dotnet-coverage`
D-10	Dependency-Check sin clave de NVD	Media	**Abierto** (bloqueado)
D-11	Sonar marcaba `allowBackup` y HTTP en la app móvil	Media	Corregido: `allowBackup=false`, cleartext solo a `10.0.2.2`
D-12	Controlador de plantilla `WeatherForecast` sin uso	Baja	Corregido: eliminado

Ningún defecto crítico o alto sigue abierto.

7. Riesgos residuales

Riesgo	Tratamiento
Fortify no está disponible; se usó Semgrep	Pedir al docente una excepción académica expresa

Riesgo	Tratamiento
La app no se probó en un teléfono físico	Requiere la IP de la PC en `AppConfig.cs` y en la `network_security_config`
Terraform y Ansible sin ejecutar contra Azure	Requiere confirmar Azure for Students
Jenkins no está montado: el `Jenkinsfile` tiene sintaxis verificada pero no se corrió en un servidor	Montar Jenkins con los agentes `qa` y `android` y las credenciales indicadas en el archivo
Sin OpenAPI publicado: ZAP API solo analiza `/health`	Publicar la especificación en QA

8. Decisión GO / NO-GO

Criterio de salida	Estado
Cero pruebas críticas fallidas	Cumple (77 / 77)
Quality Gate aprobado sin cambiar umbrales	Cumple
Sin vulnerabilidades críticas no aceptadas	Cumple en lo medido (Dependency-Check pendiente)
Reportes obligatorios presentes	**No cumple**: falta Dependency-Check
p95 < 500 ms y errores < 1 % a escala completa	**No demostrado**: solo corridas cortas
Usabilidad ≥ 80 % con 5 participantes	**No demostrado**: sin participantes
Pipeline de Jenkins ejecutado con el reporte de Quality Gate	**No demostrado**: Jenkins sin montar
Autorización explícita de Alejandro en Jenkins	Pendiente

Decisión recomendada: NO-GO provisional. Para pasar a GO hay que cerrar, en este orden:

1. Solicitar la clave gratuita de NVD y ejecutar Dependency-Check (Andrea).
2. Correr load, stress y spike a escala completa en la máquina dedicada (Andrea).
3. Hacer la sesión de usabilidad con 5 personas y registrar `docs/usabilidad/plantilla-resultados.csv` (Manuel).
4. Montar Jenkins y correr el pipeline completo con el candidato (Manuel y Alejandro).
5. Aprobar o rechazar los riesgos residuales de la sección 7; después Alejandro crea la etiqueta del candidato y autoriza el despliegue.

9. Evidencia y reproducibilidad

- Matriz de riesgos: `docs/matriz-riesgos.xlsx` (14 riesgos con probabilidad, impacto y pruebas asociadas).
- Casos de prueba con trazabilidad a requisito, PR, prueba, resultado, evidencia y defecto: `docs/casos-prueba.xlsx` (hojas Casos, Requisitos y Defectos).
- Plan de pruebas y arquitectura: `docs/plan-pruebas.pdf`, `docs/arquitectura.pdf`.
- Cobertura: `bash tests/coverage/run-coverage.sh` y `bash tests/coverage/sonar-scan.sh`. Reportes en `reports/` (ignorada por Git; cada ejecución del pipeline los archiva por número de build y

commit).
- Regeneración de documentos: `node docs/build-pdfs.mjs` y `python docs/src/build-xlsx.py`.